SOFTWARE REQUIREMENTS SPECIFICATION

G2 — Data & Intelligence

OnTime Public Transport System

Version	1.1 (Updated)
Date	April 11, 2026
Status	Draft
Previous	SRS v1.0 (April 7, 2026)

1. Introduction

1.1 Purpose

This SRS defines the complete functional and non-functional requirements for the G2 - Data & Intelligence subsystem. Version 1.1 expands the original document with full user scenarios for all three system actors (Passenger, Driver, Scheduler), use-case specifications, and incremental scope mapping.

1.2 Scope

G2 is ingesting GPS data from G1 edge devices, processing it through ML pipelines, predicting ETAs, detecting anomalies, managing scheduling logic, and exposing results via REST/WebSocket APIs to G3 frontends.

Operational scope:

- MVP Route: Moratuwa Bus Stand → Kadawatha Bus Stand
  - Segment A (Urban): Moratuwa → Kahathuduwa Entrance
  - Segment B (Highway): Kahathuduwa → Kadawatha Exit
  - Segment C (Urban): Kadawatha Exit→ Kadawatha bus stand
- Multi-route: Architecturally supported via `route_id`; operational in Increment 5 via GTFS import.

1.3 Definitions & Abbreviations

Term	Meaning
ETA	Estimated Time of Arrival
GPS	Global Positioning System
MQTT	Message Queuing Telemetry Transport
Kafka	Apache Kafka: distributed event streaming
Flink	Apache Flink: stream processing framework
PostGIS	Spatial extension for PostgreSQL
XGBoost	Extreme Gradient Boosting (ML algorithm)
GTFS	General Transit Feed Specification
Geofence	Virtual geographic boundary defined by coordinates
Crowd Band	Color-coded occupancy indicator (Green/Yellow/Red)
DLQ	Dead Letter Queue

1.4 References

- SRS v1.0 (docs/srs/SRS_G2_Data_Intelligence_1.0.docx)
- Group G Project Brief (April 2026)
- IEEE Std 830-1998

- Apache Kafka / Flink / FastAPI / PostGIS / XGBoost documentation

1.5 Change Log from v1.0

Section	Change
2.4	Expanded from 6 actors to full 3-role user model
3	New: 10 user stories (previously 4)
4	New: Scheduling FRs (FR-7.x), Driver Management FRs (FR-8.x), Route Search FRs (FR-9.x)
5	Added scenario-based use cases for all three roles
6.1	Crowd data interface marked TBD (was simulated)
7	New: Bus state machine, departure slot schema, driver issue schema

2. Overall System Description

2.1 System Context

G2 sits between hardware edge (G1) and user-facing apps (G3), with infrastructure managed by G4. G2 has no direct user interface. It is a backend consumed through APIs.

```
G1 (Edge/IoT) ──MQTT──▶ G2 (Data & Intelligence) ──REST/WS──▶ G3 (UI/UX)
                                    │
                              G4 (Platform/Security)
```

2.2 Product Functions

Function	Description
GPS Stream Processing	Ingest, clean, enrich 1 Hz GPS from active buses
Feature Engineering	Extract 16 ML features per GPS point
ETA Prediction	Predict per-stop arrival with confidence score
Anomaly Detection	3-layer detection (statistical + ML + rules)
Geofencing	Highway mode switch, boarding cutoff
Route Management	CRUD for routes/stops, GTFS import, search
Trip Scheduling	Departure slot grid, bus availability, dispatch
Driver Management	Bus state machine, issue reporting
REST API + WebSocket	Expose all data to G3
Historical Storage	Persist trips/GPS for model retraining

2.3 User Classes and Characteristics

Actor	Platform	Auth	Interaction with G2
Passenger	Mobile (Flutter)	None (public data)	Indirect; G3 consumes G2 APIs
Bus Driver	Mobile (Flutter)	Bus-level credentials (Keycloak JWT)	Status changes, issue reports via G3→G2 API
Scheduler	Web (Next.js)	Username/password (Keycloak JWT)	Dispatch assignments, fleet monitoring via G3→G2 API
G1 Edge Device	IoT hardware	MQTT credentials + API key	GPS data via MQTT
G3 Frontend	Mobile/Web app	X-API-Key	REST + WebSocket consumer
G4 Platform	Infrastructure	None	Health/metrics endpoints

2.4 Operating Environment

- Python 3.12, Docker, Kubernetes (G4)
- Apache Kafka 7.6, Apache Flink (PyFlink)
- PostgreSQL 16 + PostGIS 3.4, Redis
- Ubuntu 22.04 LTS

2.5 Design Constraints

- CON-1: All services containerized via Docker (G4 standards)
- CON-2: Inter-group communication via defined contracts (MQTT/REST schemas)
- CON-3: No hardcoded secrets; environment variables only
- CON-4: Occupancy data simulated for MVP; hardware integration TBD
- CON-5: Single route (Moratuwa→Kadawatha) for Increment 1
- CON-6: GPS coordinates validated within Sri Lanka bounding box
- CON-7: Authentication managed by G4 (Keycloak); G2 validates JWT only

2.6 Assumptions & Dependencies

- G1 provides GPS at 1 Hz in agreed MQTT JSON format
- G4 provides running Kafka, PostgreSQL, Keycloak infrastructure
- Historical trip data synthetically generated or from GPS simulator
- G3 handles all UI rendering; G2 provides data only
- Route data sourced from GTFS feeds (for multi-route increments)

3. User Stories & Acceptance Criteria

3.1 Passenger Stories

US-P1: Live Bus Tracking High Priority — Increment 1

As a passenger, I want to see buses moving on the map in real-time, so I know if a bus is approaching.

Acceptance Criteria	G2 Obligation
Bus positions update within 5 seconds of real-world	WebSocket `/live-feed` delivers within 1s of GPS receipt
Route line drawn on map in grey	`GET /routes/{id}` returns GeoJSON geometry
Bus dots show on route line	Live feed includes lat/lng per bus

US-P2: ETA on Tap High Priority — Increment 2
As a passenger, I want to tap a bus stop and see when the next bus arrives, so I can plan my wait.

Acceptance Criteria	G2 Obligation
ETA shows in minutes with confidence	`GET /eta/{bus_id}/{stop_id}` returns `eta_seconds` + `confidence`
ETA updates dynamically	Prediction refreshes every ~5s via Flink cycle
Fallback when ML unavailable	Physics heuristic returns `model_version: "heuristic"`

US-P3: Route Search Medium Priority — Increment 5
As a passenger, I want to search by route number or destination, so I can find which bus to take.

Acceptance Criteria	G2 Obligation
Search by route number returns route	`GET /routes/search?q=138` returns matching routes
Search by destination works	Text search on stop names along routes
Direction swap (outbound/inbound)	Route variants returned with direction flag
Selected route highlights on map	Route GeoJSON served with direction metadata

US-P4: Nearest Route Discovery Low Priority — Increment 5
As a passenger with location enabled, I want the app to show the nearest routes and stops automatically.

Acceptance Criteria	G2 Obligation
Nearest stops within 500m shown	`GET /routes/nearest?lat=X&lng=Y&radius=500` PostGIS spatial query
Nearest route highlighted differently	Response includes distance + route info

3.2 Driver Stories

US-D1: Bus Login & Trip Start High Priority — Increment 1
As a bus driver, I want to log in as my bus and start a trip, so the system begins tracking.

Acceptance Criteria	G2 Obligation
Login with bus credentials	Keycloak (G4) validates; G2 receives JWT with bus_id claim
Tap "Start Trip" → bus appears on map	`POST /bus/{bus_id}/status` body: `{status: "DEPARTED_ORIGIN"}`
Tap "End Trip" → bus marked arrived	Status transition to `ARRIVED_DESTINATION`

US-D2: Target Time Visibility Medium Priority — Increment 2

As a driver, I want to see dynamic target times for upcoming checkpoints, so I know if I'm on schedule.

Acceptance Criteria	G2 Obligation
Next checkpoint + target time shown	ETA prediction for next major stop returned to driver UI
Updates dynamically based on current speed	Same ETA engine as passenger, served to driver via API

US-D3: Set Availability Medium Priority — Increment 3

As a driver, after completing a trip, I want to tell the system when I'll be available for the next assignment.

Acceptance Criteria	G2 Obligation
Set "Available in X minutes"	`POST /bus/{bus_id}/availability` body: `{available_at: ISO8601}`
Scheduler sees bus in available pool after window	Scheduling service query filters by availability time

US-D4: Report Issue Medium Priority — Increment 4

As a driver, I want to report a breakdown or traffic jam with one tap, so dispatch knows immediately.

Acceptance Criteria	G2 Obligation
Big-button issue reporting	`POST /bus/{bus_id}/issue` with enum type
Issue types: BREAKDOWN, ACCIDENT, HEAVY_TRAFFIC, ROAD_BLOCKED, WEATHER, OTHER	Validated enum in Pydantic schema
"Terminate Trip" option	Status → TERMINATED; event on `bus.status` topic; Scheduler alerted

3.3 Scheduler Stories

US-S1: Fleet Overview High Priority — Increment 1

As a scheduler, I want to see all active buses on a live map, so I have fleet visibility.

Acceptance Criteria	G2 Obligation
Map shows all buses with status colors	WebSocket fleet feed includes all buses with status field
Bus details on click	`GET /routes/{route_id}/buses` returns full bus info

US-S2: Dispatch Bus to Slot High Priority — Increment 3
As a scheduler, I want to assign an available bus to the next departure slot, so the route stays served.

Acceptance Criteria	G2 Obligation
View departure slot grid	`GET /schedule/{route_id}/slots` returns time slots with assignment status
See available buses	`GET /buses/available?route_id=X` returns buses past their availability window
Assign bus to slot	`POST /schedule/assign` body: `{bus_id, slot_id}`
Prevent double-booking	409 Conflict if slot already assigned
MVP: only next 1 slot assignable	Business rule enforced in service

US-S3: Anomaly Dashboard Medium Priority — Increment 4
As a scheduler, I want to see real-time anomaly alerts, so I can take corrective action.

Acceptance Criteria	G2 Obligation
Active anomalies listed with severity	`GET /anomalies/active` returns all unresolved
Terminated buses highlighted	Bus status = TERMINATED flagged in fleet feed
Alert includes recommended action	Anomaly record includes `recommended_action` field

4. Functional Requirements

4.1 Data Ingestion (Increment 1)

ID	Requirement	Priority
FR-1.1	Consume GPS from Kafka topic `gps.raw.{bus_id}` via MQTT-to-Kafka bridge	Must Have
FR-1.2	MQTT bridge subscribes to `gps/bus/+` and produces validated JSON to Kafka at 1 Hz	Must Have
FR-1.3	Reject invalid GPS to dead letter topic `gps.dlq` (schema fail or out-of-bounds)	Must Have
FR-1.4	Persist raw + cleaned GPS to `gps_readings` table partitioned by date	Must Have

4.2 Stream Processing & Feature Engineering (Increments 1–2)

ID	Requirement	Priority
FR-2.1	Apply Kalman filter to reduce positional noise	Must Have
FR-2.2	Map-match GPS to nearest route segment	Must Have
FR-2.3	Extract 16 ML features per GPS point (see §7.2)	Must Have
FR-2.4	Flink tumbling window: 5s with 3s watermark	Must Have
FR-2.5	Publish enriched features to `gps.features`	Must Have

4.3 ETA Prediction (Increment 2)

ID	Requirement	Priority
FR-3.1	Predict remaining travel time in seconds per downstream stop	Must Have
FR-3.2	Use XGBoost regressor as primary algorithm	Must Have
FR-3.3	Fall back to physics heuristic when confidence < 0.5	Must Have
FR-3.4	Switch urban → highway model at Kahathuduwa geofence	Must Have
FR-3.5	Refresh predictions every Flink cycle (~5s)	Must Have
FR-3.6	Support batch retraining via `train_models.py`	Should Have

4.4 Anomaly Detection (Increment 4)

ID	Requirement	Priority
FR-4.1	Layer 1 (Statistical): Z-score on speed/dwell/heading; warn >2.5, critical >3.5	Must Have
FR-4.2	Layer 2 (ML): Isolation Forest on 16 features; contamination=0.05	Should Have
FR-4.3	Layer 3 (Rules): off-route >200m/30s, stopped >5min non-stop, GPS loss >60s, speed violations	Must Have
FR-4.4	Aggregate 3 layers into unified anomaly with confidence	Must Have
FR-4.5	Persist anomalies to PostgreSQL; publish to `alerts.anomaly`	Must Have
FR-4.6	Mark anomalies resolved when condition clears	Should Have

4.5 REST API & WebSocket (All Increments)

ID	Requirement	Priority	Increment
FR-5.1	`GET /eta/{bus_id}` — per-stop ETAs	Must Have	2
FR-5.2	`GET /eta/{bus_id}/{stop_id}` — specific stop ETA	Must Have	2
FR-5.3	`GET /anomalies/{bus_id}` — last 10 anomalies	Must Have	4
FR-5.4	`GET /anomalies/active` — all unresolved	Must Have	4
FR-5.5	`POST /ingest/gps` — test endpoint	Should Have	1
FR-5.6	`GET /routes/{route_id}/buses` — active buses on route	Must Have	1

ID	Requirement	Priority	Increment
FR-5.7	`WS /live-feed` — 1s fleet status push	Must Have	1
FR-5.8	`GET /health` — dependency status	Must Have	0
FR-5.9	`GET /metrics` — Prometheus format	Must Have	0

4.6 Geofencing & Boarding (Increment 2)

ID	Requirement	Priority
FR-6.1	Persist Kahathuduwa geofence polygon in PostGIS	Must Have
FR-6.2	Set bus status `boarding_unavailable` inside the geofence	Must Have
FR-6.3	Reset to `active` when bus exits geofence	Must Have
FR-6.4	Spatial query: "buses between Stop A and Stop B"	Should Have

4.7 Trip Scheduling & Dispatch (Increment 3) — NEW

ID	Requirement	Priority
FR-7.1	Maintain pre-defined departure slot grid per route (configurable intervals)	Must Have
FR-7.2	`GET /schedule/{route_id}/slots` — list slots with assignment status	Must Have
FR-7.3	`POST /schedule/assign` — assign available bus to next slot; reject if occupied (409)	Must Have
FR-7.4	`GET /buses/available` — list buses past their availability window	Must Have
FR-7.5	MVP: Scheduler can only assign the next 1 upcoming departure slot per route	Must Have
FR-7.6	Publish dispatch events to `schedule.dispatch` Kafka topic	Should Have
FR-7.7	Departure slot seed script for MVP route (15-min intervals, 05:00–22:00)	Must Have

4.8 Driver & Bus Management (Increments 1, 3, 4) — NEW

ID	Requirement	Priority	Increment
FR-8.1	Bus state machine: `IDLE → WAITING_AT_DEPOT → DEPARTED_ORIGIN → EN_ROUTE → ARRIVED_DESTINATION → IDLE`	Must Have	1
FR-8.2	`POST /bus/{bus_id}/status` — driver changes state (manual tap)	Must Have	1
FR-8.3	EN_ROUTE set automatically when GPS data flows after DEPARTED_ORIGIN	Should Have	1
FR-8.4	`POST /bus/{bus_id}/availability` — set available_at timestamp	Must Have	3
FR-8.5	`POST /bus/{bus_id}/issue` — report issue with enum type	Must Have	4

ID	Requirement	Priority	Increment
FR-8.6	Issue enum: `BREAKDOWN, ACCIDENT, HEAVY_TRAFFIC, ROAD_BLOCKED, WEATHER, OTHER`	Must Have	4
FR-8.7	TERMINATED status: removes from fleet, publishes event, alerts Scheduler	Must Have	4
FR-8.8	Publish all state changes to `bus.status` Kafka topic	Must Have	1

4.9 Route Search & Discovery (Increment 5) — NEW

ID	Requirement	Priority
FR-9.1	`GET /routes/search?q={query}` — search by route number, origin name, destination name	Must Have
FR-9.2	Each route has two direction variants (outbound + inbound) with direction flag	Must Have
FR-9.3	`GET /routes/nearest?lat=X&lng=Y&radius=N` — spatial query for nearest routes/stops	Should Have
FR-9.4	GTFS import script to seed route geometry and stops from GTFS feed	Must Have
FR-9.5	Route search returns stops with `is_highway_entry` flag	Should Have

5. Use Case Scenarios

5.1 UC-1: Passenger Views Live Bus (Increment 1)

Step	Actor	Action	System Response
1	Passenger	Opens app (location on/off)	Map loads centered on Colombo area
2	Passenger	Sees route line on map (grey)	Route geometry loaded from G2 API
3	System	—	WebSocket connects; bus dots appear on route
4	Passenger	Watches bus dot move	Position updates every 1 second

- Preconditions: At least one bus has active GPS.
- Postconditions: Passenger sees real-time bus positions.

5.2 UC-2: Passenger Checks ETA (Increment 2)

Step	Actor	Action	System Response
1	Passenger	Taps a bus stop on the map	G3 calls `GET /eta/{bus_id}/{stop_id}`
2	System	—	Returns ETA in seconds + confidence score
3	Passenger	Sees "Bus arriving in ~4 min"	ETA updates dynamically every 5s

- Alternative: If ML model unavailable → physics heuristic with lower confidence displayed.

5.3 UC-3: Passenger Searches Route (Increment 5)

Step	Actor	Action	System Response
1	Passenger	Types "138" or "Kadawatha" in search	G3 calls `GET /routes/search?q=138`
2	System	—	Returns matching route(s) with direction variants
3	Passenger	Selects route	Route highlights on map (Blue=outbound, Green=inbound)
4	Passenger	Taps direction swap	Colors swap; stops show for other direction
5	Passenger	Taps "Cancel"	Route deselects; map returns to default

5.4 UC-4: Driver Starts Trip (Increment 1)

Step	Actor	Action	System Response
1	Driver	Opens app, enters bus credentials	Keycloak validates → JWT issued
2	Driver	Sees control panel with big buttons	Status: IDLE
3	Driver	Taps "Arrived at Platform"	Status → WAITING_AT_DEPOT
4	Driver	Taps "Depart"	Status → DEPARTED_ORIGIN → bus appears on map
5	System	GPS flows	Status auto-transitions to EN_ROUTE
6	Driver	Arrives at destination, taps "Trip Complete"	Status → ARRIVED_DESTINATION → IDLE

5.5 UC-5: Driver Reports Issue (Increment 4)

Step	Actor	Action	System Response
1	Driver	Taps "Report Issue"	Category selection shown (big buttons)
2	Driver	Selects "Breakdown"	Issue recorded; anomaly created
3	Driver	Optionally taps "Terminate Trip"	Bus status → TERMINATED
4	System	—	Scheduler receives alert; bus removed from active fleet

5.6 UC-6: Scheduler Dispatches Bus (Increment 3)

Step	Actor	Action	System Response
1	Scheduler	Opens web dashboard	Fleet map loads; all active buses visible
2	Scheduler	Selects route from dropdown	Departure slot grid appears
3	Scheduler	Views next available slot (e.g., 10:15 AM)	Slot shown as "Open"

Step	Actor	Action	System Response
4	Scheduler	Views available bus pool	Buses past their availability window listed
5	Scheduler	Assigns Bus-007 to 10:15 AM slot	Assignment confirmed; driver notified via app

- Error Case: Slot already assigned → 409 Conflict → "Slot already occupied."

6. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	End-to-end GPS→API latency ≤ 2.0 seconds
NFR-2	Performance	REST p95 latency < 100ms
NFR-3	Performance	WebSocket push at 1s intervals, jitter < 200ms
NFR-4	Performance	Map lag < 5s behind real-world position
NFR-5	Scalability	Support 50 concurrent buses without re-architecture
NFR-6	Scalability	Stateless API for horizontal scaling
NFR-7	Reliability	MQTT bridge reconnects within 10s
NFR-8	Reliability	GPS gaps < 30s handled via interpolation
NFR-9	Reliability	99% uptime during 05:00–23:00
NFR-10	Security	GPS ingestion requires X-API-Key
NFR-11	Security	Public endpoints rate-limited: 60 req/min/IP
NFR-12	Security	Parameterized queries (ORM) only
NFR-13	Security	No hardcoded credentials
NFR-14	Security	Driver/Scheduler auth via Keycloak JWT
NFR-15	Maintainability	Type hints + Pydantic models on all I/O
NFR-16	Maintainability	pytest coverage ≥ 70%
NFR-17	Portability	All services run in Docker containers
NFR-18	Usability	Driver UI: maximum 3 taps for any action
NFR-19	Offline	Passenger app shows "No connection" when offline; route geometry cached

7. Data Requirements

7.1 GPS Reading Schema

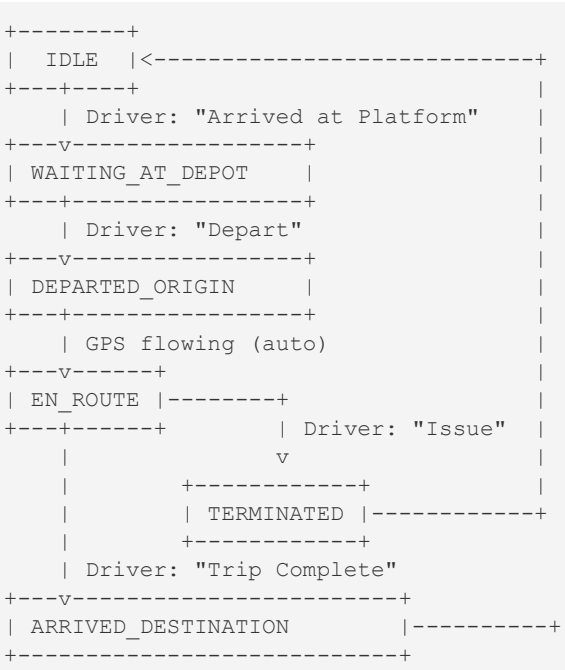
(Unchanged from v1.0 — see §7.1)

Field	Type	Example
bus_id	string	BUS-001
lat	float (WGS84)	6.7736
lng	float (WGS84)	79.8820
speed	float (km/h)	35.2
heading	float (°)	45.0
timestamp	ISO 8601	2026-04-07T08:30:00Z
route_id	string	MOR-KAD-01

7.2 ML Feature Set (16 Features)

(Unchanged from v1.0 — see §7.2)

7.3 Bus State Machine — NEW



7.4 Departure Slot Schema — NEW

Field	Type	Example
slot_id	UUID	550e8400-...
route_id	string	MOR-KAD-01
direction	enum	OUTBOUND / INBOUND
scheduled_time	time	10:15:00
assigned_bus_id	string (nullable)	BUS-007

Field	Type	Example
status	enum	OPEN / ASSIGNED / DEPARTED / COMPLETED

7.5 Driver Issue Schema — NEW

Field	Type	Example
issue_id	UUID	—
bus_id	string	BUS-001
issue_type	enum	BREAKDOWN
description	string (optional)	"Engine overheated"
location	PostGIS POINT	(6.74, 80.02)
timestamp	ISO 8601	2026-04-07T08:35:00Z
resolved	boolean	false

7.6 Bus Availability Schema — NEW

Field	Type	Example
bus_id	string	BUS-007
available_at	ISO 8601	2026-04-07T10:00:00Z
set_at	ISO 8601	2026-04-07T09:30:00Z
current_location	PostGIS POINT	(6.79, 79.90)

7.7 Crowd Data Interface — TBD

> ⚠ *Pending G1 IR sensor implementation.*
> The MQTT topic, payload schema, and processing pipeline for crowd/occupancy data will be defined in a future SRS revision once G1 confirms their sensor design.

8. External Interfaces

8.1 G1 → G2 (Input)

Unchanged from v1.0 — MQTT on `gps/bus/{bus_id}` at 1 Hz

8.2 G2 → G3 (Output)

Unchanged from v1.0 — REST + WebSocket, X-API-Key auth

8.3 G2 ↔ G4 (Platform)

- Docker images built by G4 CI/CD
- Health/metrics endpoints for Prometheus
- NEW: Keycloak JWT validation for driver/scheduler endpoints

9. System Architecture

9.1 Microservices Overview

Service	Responsibility
Ingestion Service	MQTT→Kafka bridge, validation, DLQ
Stream Processing	Flink: Kalman filter, map matching, features
ETA Prediction Service	XGBoost inference, physics fallback
Anomaly Detection Service	3-layer detection, result aggregation
Route Management Service	CRUD routes/stops, GTFS import, search
Scheduling Service	Departure slots, availability, dispatch
API Gateway	FastAPI: REST + WebSocket + caching

9.2 Database Tables

Table	Purpose
routes	Route definitions with PostGIS LINESTRING
stops	Stops with POINT geometry, sequence, highway flags
buses	Fleet registry with status and assigned route
gps_readings	GPS points partitioned by date
trips	Journey records for ML training
stop_arrivals	Actual vs scheduled arrival times
anomalies	Anomaly records with location and severity
geofences	Polygon regions (highway entry/exit)
departure_slots	Pre-defined time slots per route — NEW
dispatch_assignments	Bus-to-slot assignments — NEW
bus_availability	Driver-set availability windows — NEW
driver_issues	Reported issues with type and location — NEW

10. Constraints & Edge Cases

10.1 Highway Mode Switch

When bus enters Kahathuduwa geofence: switch to highway model, set `boarding_unavailable`. Revert at Gelanigama exit.

10.2 GPS Signal Loss

- Gap < 30s: Interpolate using last speed + historical segment speed
- Gap 30–60s: Confidence → 0.3; include uncertainty
- Gap > 60s: Status → `unknown`; fire `signal_gap` anomaly (critical)

10.3 Cold Start (No Trained Model)

Physics heuristic exclusively. API returns `model_version: "heuristic"`. Training triggered manually after data accumulation.

10.4 Driver Not Tapping Status

If GPS shows bus in motion but status is still IDLE — system does NOT auto-correct in MVP. Future increment may add smart detection.

10.5 Schedule Conflicts

If a bus is assigned to a slot but doesn't depart within 10 minutes of scheduled time, the slot status remains ASSIGNED (no auto-cancellation in MVP).

11. Appendix — Complete API Endpoint Summary

Method	Path	Increment	Consumer
GET	/health	0	All
GET	/metrics	0	G4
GET	/api/v1/routes	1	G3
GET	/api/v1/routes/{route_id}/buses	1	G3
WS	/ws/live-feed	1	G3
POST	/api/v1/bus/{bus_id}/status	1	G3 (Driver)
POST	/api/v1/ingest/gps	1	G1/Test
GET	/api/v1/eta/{bus_id}	2	G3
GET	/api/v1/eta/{bus_id}/{stop_id}	2	G3
GET	/api/v1/schedule/{route_id}/slots	3	G3 (Scheduler)
GET	/api/v1/buses/available	3	G3 (Scheduler)
POST	/api/v1/schedule/assign	3	G3 (Scheduler)
POST	/api/v1/bus/{bus_id}/availability	3	G3 (Driver)
GET	/api/v1/anomalies/{bus_id}	4	G3

Method	Path	Increment	Consumer
GET	/api/v1/anomalies/active	4	G3 (Scheduler)
POST	/api/v1/bus/{bus_id}/issue	4	G3 (Driver)
GET	/api/v1/routes/search	5	G3
GET	/api/v1/routes/nearest	5	G3

End of SRS v1.1 — G2 Data & Intelligence